

Лабораторная работа	М3235	Программирование на видеокартах
PrefSum.CUDA	Перепелкин Богдан Алексеевич	2026

kogge stone

Это наивная реализация, в ней не используется факт того, что часть работы можно гарантированно вычислять строго внутри варпа и делать за счет этого меньше синхронизаций между всеми тредами в группе. Тут много барьеров (очень похоже на пр-1), поэтому скорость печальна, на самых результативных конфигах хуже чем обычный линейный подсчет на сри в 2 раза (для размера 300 млн элементов)

замеры времени в зависимости от block size:

TIME = 391.676971, block size = 32

TIME = 712.867676, block size = 64

TIME = 814.894348, block size = 128

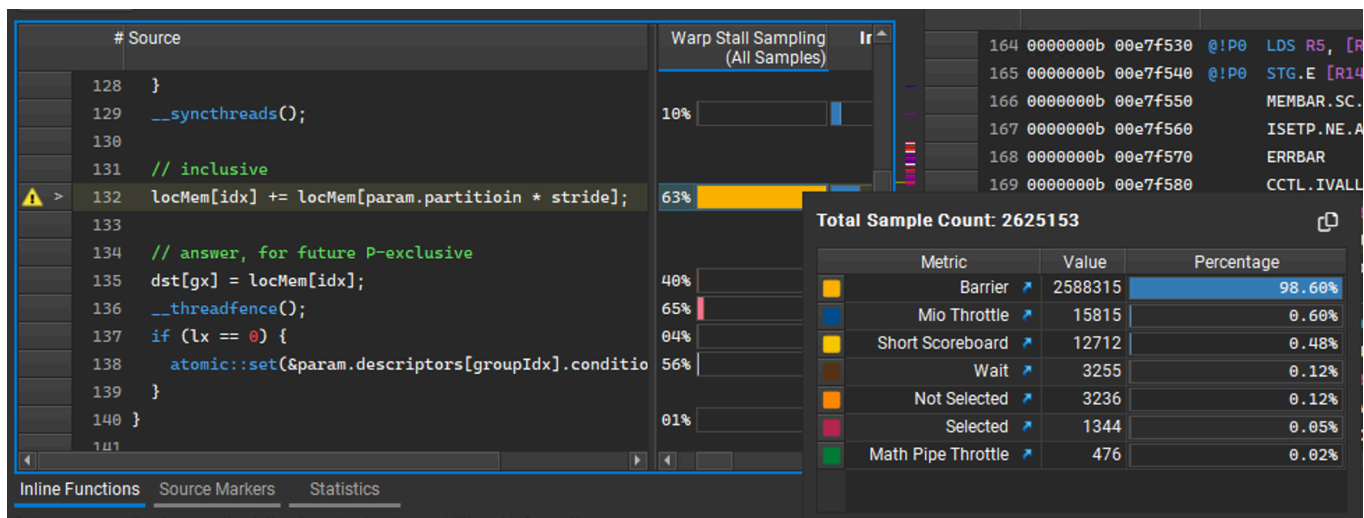
TIME = 889.286011, block size = 256

TIME = 995.266235, block size = 512

TIME = 1099.517212, block size = 1024

про adaptive look-back

у алгоритма из статьи есть особенность - он сильно завязан на межгрупповой синхронизации таким образом, что достаточно 1 варпа в группе ,чтобы посчитать exclusive sum. Из этого следует, что лишь 1 варп в группе будет не целиком висеть на барьере, а остальные будут на нем целиком и это хорошо, так как они целиком спят и можно исполнить кого-то другого. Эта особенность диктует бутылочное горлышко, которое я подтвердил экспериментально:



видно, в строке

```
locMem[lx] += locMem[lockSize];
```

69.52 % варпов в stall на барьере, эта характеристика показывает процент варпов, в которых каждый тред встал на барьере. И то что их почти 70 % говорит об успешной реализации алгоритма CUB, ведь значит что бо́льшая часть варпов повисла на барьере и лишь 30 % вынуждены на нем ждать неэффективно.

Это же является причиной хорошего фактического ossurance, ведь целиком спящие потоки можно убирать с исполнения и ставить на их место готовые.

комментарий к CUB

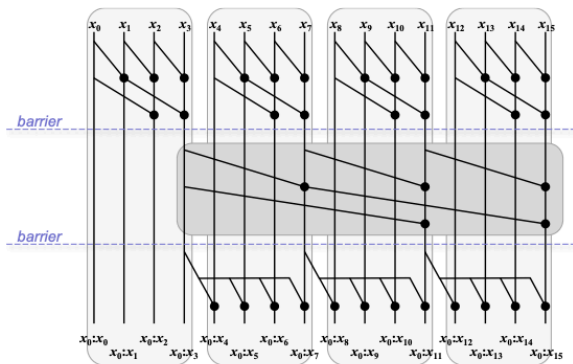
(речь идет про межгрупповое взаимодействие)

Алгоритм имеет бутылочное горлышко - ожидание всех потоков на синхронизации в то время как один поток ходит и ищет у предшественников exclusive sum. Я думал как это оптимизировать, но пришел к выводу что с данной идеей ничего лучше не сделать, так как любые попытки перераспределить работу глобального (global memory) хождения по предшественникам иначе как в моей реализации просто приведут к такому же количеству синхронизаций либо бо́льшему без видимых предпосылок к увеличению производительности.

brentstone:

*(как я его сокращенно называю, полное - Brent-Kung scan-then-propagate strategy embedding Kogge-Stone warpscan and propagation fans)

Переход с kogge stone на brent stone обусловлен тем, что второй алгоритм, позволяет организовать работу так, что понадобится меньше межварповых синхронизаций, а также сократит общее количество операций относительно kogge stone. В результате количество операций `__syncthreads()` сократилось в $O(n)$ раз, им на смену пришли внутриварповые, которые дешевле. Я взял идею из алгоритма в пункте 2а и реализовал ее так, что можно задавать размер разбиения не равный размеру kogge-stone на нижнем уровне. Это позволяет перебрать существенно больше сценариев, среди которых в топе по производительности как моно заметить как раз такие представители, у которых $partition \neq low\ range$, где $partition$ - размер разбиения (для 2-го уровня), $low\ range$ - сколько потоков нужно для нижнего kogge-stone



(a) A radix-4 Brent-Kung *scan-then-propagate* strategy embedding Kogge-Stone warps and propagation fans

Поиск лучшей конфигурации в зависимости от block size и partition:

TIME = 25.744480, block size = 1024, partition = 32, low range = 32
 TIME = 26.794657, block size = 512, partition = 32, low range = 16
 TIME = 35.591904, block size = 512, partition = 16, low range = 32
 TIME = 36.719360, block size = 256, partition = 32, low range = 8
 TIME = 36.723137, block size = 256, partition = 16, low range = 16
 TIME = 36.862720, block size = 256, partition = 8, low range = 32
 TIME = 50.957855, block size = 128, partition = 16, low range = 8
 TIME = 50.967232, block size = 128, partition = 8, low range = 16
 TIME = 51.201794, block size = 128, partition = 4, low range = 32
 TIME = 72.289246, block size = 64, partition = 4, low range = 16
 TIME = 72.312355, block size = 64, partition = 8, low range = 8
 TIME = 72.332832, block size = 64, partition = 16, low range = 4
 TIME = 139.180679, block size = 32, partition = 1, low range = 32
 TIME = 145.791748, block size = 32, partition = 2, low range = 16
 TIME = 145.978470, block size = 32, partition = 8, low range = 4
 TIME = 148.683167, block size = 32, partition = 4, low range = 8

occupancy

Интересная статистика по occupancy в зависимости от алгоритма

kogge stone:

Theoretical Occupancy [%]	50
Theoretical Active Warps per SM [warp]	24
Achieved Occupancy [%]	49.93
Achieved Active Warps Per SM [warp]	23.97

brent stone (block size = 1024, partition = 32):

Theoretical Occupancy [%]	66.67
Theoretical Active Warps per SM [warp]	32
Achieved Occupancy [%]	65.15
Achieved Active Warps Per SM [warp]	31.27

виден прирост occupancy, а учитывая, что остальные проблемы (ожидание на барьере) не сильно поменялись, можно сделать вывод, что в данном случае дал фактический прирост к быстродействию.

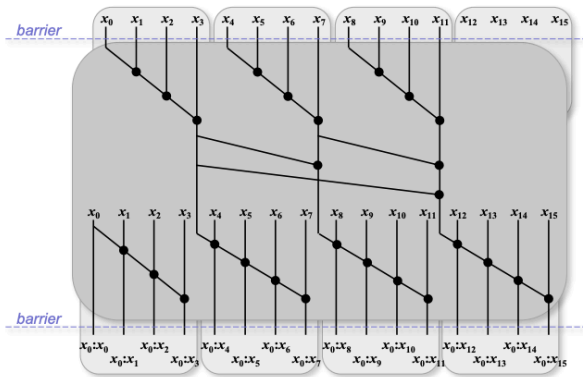
brent stone если попробовать подобрать другие параметры для brent stone (block size = 512, partition = 16):

Theoretical Occupancy [%]	100
Theoretical Active Warps per SM [warp]	48
Achieved Occupancy [%]	97.72
Achieved Active Warps Per SM [warp]	46.91

occupancy сильно возрастает, но по факту прироста к времени работы не дает, то есть я обнаружил ситуацию, когда повышение occupancy дает прирост к быстродействию только до какого-то момента, а затем не влияет.

reduce stone

Это моя реализация алгоритма 2a внутриварпового взаимодействия из статьи. Он очень похож на brent stone, а отличается тем, что на нижнем уровне делает меньшее число операций и работает за n , а не $\log$ от размера нижнего уровня. Время работы и результаты профилировки идентичны.



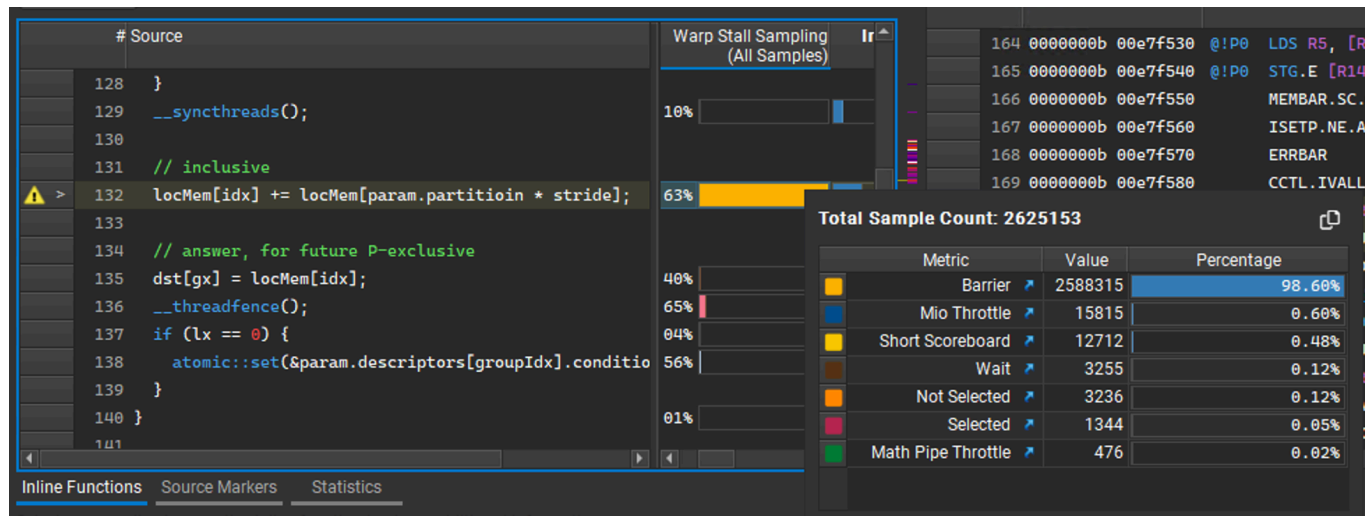
(b) A raking radix-4 *reduce-then-scan* strategy embedding serial reductions, serial scans, and a Kogge-Stone warpscan at the root

замеры времени в зависимости от block size

TIME = 27.257696, block size = 512
 TIME = 27.894049, block size = 512
 TIME = 27.932608, block size = 1024
 TIME = 36.884575, block size = 256
 TIME = 37.777985, block size = 256
 TIME = 38.152481, block size = 256
 TIME = 51.311935, block size = 128
 TIME = 52.091358, block size = 128
 TIME = 55.952225, block size = 128
 TIME = 72.177536, block size = 64
 TIME = 72.249664, block size = 64
 TIME = 72.344894, block size = 64
 TIME = 141.142334, block size = 32
 TIME = 146.796890, block size = 32
 TIME = 149.227997, block size = 32
 TIME = 160.590591, block size = 32

bank conflicts

посмотрел на это



решил что возможно здесь есть банк-конфликты, из-за которых появляется долгое обращение к памяти, из-за чего получается барьер, следовательно замедление и все вытекающие из профайлера вещи. Для этого я сделал наивное решение - искусственно перевернул логику локальной памяти в 2-D и добавил мусорные элементы после каждого слоя, чтобы сдвинуть индексацию банков. Это вообще не помогло ни по результатам профайлера, ни по времени

Поиск лучшей конфигурации в зависимости от block size и partition:

TIME = 25.470783, block size = 1024, partition = 32, low range = 32
TIME = 26.701696, block size = 512, partition = 16, low range = 32
TIME = 26.792288, block size = 512, partition = 32, low range = 16
TIME = 36.867519, block size = 256, partition = 32, low range = 8
TIME = 36.882881, block size = 256, partition = 16, low range = 16
TIME = 36.968830, block size = 256, partition = 8, low range = 32
TIME = 51.078495, block size = 128, partition = 8, low range = 16
TIME = 51.085503, block size = 128, partition = 16, low range = 8
TIME = 51.275455, block size = 128, partition = 4, low range = 32
TIME = 72.000572, block size = 64, partition = 4, low range = 16
TIME = 72.027267, block size = 64, partition = 8, low range = 8
TIME = 72.119232, block size = 64, partition = 16, low range = 4
TIME = 139.653244, block size = 32, partition = 4, low range = 8
TIME = 139.655579, block size = 32, partition = 2, low range = 16
TIME = 139.686630, block size = 32, partition = 8, low range = 4
TIME = 148.229828, block size = 32, partition = 1, low range = 32

atomic

изначально для чтения `int` из дескрипторов в глобальной памяти я использовал `atomic` функции от `nvidia`, но потом осознал, что гарантируется, что чтение и запись в `int` будет атомарной и тогда нет `data race`. Я убрал `atomic`, сравнил время работы с атомиком и без, ничего не поменялось, только шумы (тестировал только на данных 0.3 Gb, сервер был занят). Это странно, но если пытаться это объяснить - видимо компилятор понял, что я делаю бесполезные вещи и оптимизировал все.